I. BEYOND THE BASICS: WHY PYTHON FOR BIG DATA?

While Python is often criticized for being slower than C++ or Java, it has become the dominant language in AI due to its role as a "Glue Language." Core scientific libraries are written in C, while Python provides high-level abstractions to manipulate them. For engineers working with millions of data points, mastering memory management and advanced Python structures is mandatory.

1.1. List Comprehensions vs. Traditional Loops

List Comprehension is not only more concise but also significantly faster because it is optimized at the Python bytecode level.

- **Syntax:** [expression for item in iterable if condition]
- **Use Case:** Batch processing of data transformations or filtering noise from a dataset.

II. ADVANCED DATA STRUCTURES

2.1. Dictionaries and Sets (Hash Tables)

Dictionaries and Sets are built upon **Hash Tables**, allowing for O(1) average time complexity for lookups.

- **In Big Data:** Use dictionaries to build fast indexes for user IDs or labels.
- **Sets:** Essential for removing duplicates in large-scale text preprocessing (NLP).

2.2. The Collections Module: NamedTuple & Counter

- **NamedTuple:** Improves code readability by allowing field access via names instead of indices, reducing errors in feature vector manipulation.
- **Counter:** The most efficient tool for calculating word frequencies in large corpora.

III. MEMORY MANAGEMENT: GENERATORS & ITERATORS

3.1. The "Big Data" Memory Problem

When processing a 10GB dataset on a machine with 8GB of RAM, traditional list operations will trigger a MemoryError.

3.2. Generators: Lazy Evaluation

Generators process data one element at a time instead of loading the entire dataset into RAM.

- **The yield Keyword:** Unlike return, yield pauses the function and returns a value only when requested.
- **Generator Expressions:** (x**2 for x in range(1000000)) consumes nearly 0MB of RAM, regardless of the range size.

IV. FUNCTIONAL PROGRAMMING IN PYTHON

4.1. Lambda Functions, Map, and Filter

Functional programming models are highly compatible with modern Data Pipelines.

- **Map:** Applies a function to an entire column of data.
- **Filter:** Selects data points based on complex criteria.

4.2. Decorators

Decorators allow developers to "wrap" functions to add functionality, such as:

- **Logging:** Recording execution time for machine learning algorithms.
- **Timing:** Benchmarking performance of data-heavy functions.

V. BEST PRACTICES FOR DTU CAPSTONE PROJECTS

To maintain code quality in your DTU projects, follow these standards:

1. **Type Hinting:** Use def process(data: List[float]) -> float: to improve code maintainability and debugging.
2. **Virtual Environments:** Always use venv or conda to manage project-specific dependencies (PyTorch, TensorFlow, etc.).
3. **PEP 8:** Adhere to Python's official style guide to ensure team collaboration efficiency.